Functions, BOM & DOM

Execution, Scope, the Browser & the Page — Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Write the code snippets when required and observe the outputs.

Part 1 — Memory, Execution & Scope

Concept — Execution Context: Two Phases. Before running your code, JavaScript creates an **execution context** for it in two phases.

1. **Memory phase:** it scans the code and reserves space for every variable and function. `var` gets `undefined`, `let`/`const` are reserved but **cannot be used yet**, and a function declaration gets the **whole function**.
2. **Code phase:** it runs the code line by line, putting real values into variables and calling functions.

Concept — Every 'function call' creates its own 'Execution Context' (EC). Memory is allocated during its creation, after which the EC is pushed onto the **Call Stack**. JavaScript then executes the code line by line. Once the function returns, its EC is **removed** along with its local variables.

Concept — Hoisting. Because of the memory phase, declarations are known **before any line runs** — as if moved ("hoisted") to the top. `var` is hoisted as `undefined`; a function declaration is hoisted **whole**, so it can be called before it is written.

Concept — Temporal Dead Zone (TDZ). `let`/`const` are hoisted too, but **not initialized**. Until their line runs, they are in the **TDZ** — using them there throws a `ReferenceError`.

Concept — Scope. Scope decides **where a variable can be used**.

- **Global scope:** declared outside any function — usable everywhere.
- **Function (local) scope:** declared inside a function — usable only inside it.
- **Block scope:** `let`/`const` declared inside `{ }` (`if`, `for`, …) — usable only inside those braces.

Code **inside** can see variables outside, but code **outside** cannot see variables inside.

TRY THIS · LIVE SERVER + DEVTOOLS

1. Create `index.html` with `script.js` linked to it, and paste the code below into `script.js`.
2. Open `index.html` with **Live Server**, then open **Developer Tools → Sources** and select `script.js` in the file tree on the left.
3. Click line numbers to add **breakpoints** (start with line 1), then **refresh** the page. As you move through the breakpoints, keep watching the **Call Stack** and **Scope** sections.

```
console.log("----- START -----");
let a = 2;
var b = 3;
console.log(a, b);
function multiplyByTwo(n) {
    console.log("----- INSIDE FUNCTION -----");
    let result = n * 2;
    console.log("----- FUNCTION ABOUT TO RETURN RESULT -----");
    return result;
    console.log("----- FUNCTION COMPLETED -----");
}
console.log("----- MID -----");
let c = multiplyByTwo(a);
console.log(c);
console.log("----- END -----");
```

Q1. (a) Paused before line 1 ran: which names are under **Script**? Expand **Global** and find `b` and `multiplyByTwo`. What does each hold, and why are they already there?

> **Note:** `let` / `const` appear under **Script**, while `var` and functions appear under **Global** — but both are global scope.

(b) Step (`F9`) or **Resume** (`F8`) through the code, watching **Call Stack** and **Scope**. What appears when `multiplyByTwo(a)` is called, and after `return`? Which log **never printed**, and why?

TRY THIS · BROWSER CONSOLE

```
console.log(x);
var x = 5;
console.log(x);
```

Q2. What did each line print? Why did the first line **not** throw an error?

TRY THIS · BROWSER CONSOLE

```
sayHi();
function sayHi() {
  console.log("Hi!");
}
```

Q3. The function was **called before it was written**. Why did it still work?

TRY THIS · BROWSER CONSOLE

```
console.log(y);
let y = 10;
```

Q4. What **error** did you get? What is the reason behind it?

TRY THIS · BROWSER CONSOLE

```
let college = "Zenith";
function intro() {
  console.log("I study at " + college);
}
intro();
```

Q5. `college` was not declared inside `intro`. How did the function still find it?

TRY THIS · BROWSER CONSOLE

```
let city = "Pune";
function showCity() {
  let city = "Delhi";
  console.log(city);
}
showCity();
console.log(city);
```

Q6. What did each `console.log` print? Why did the `city` inside the function **not change** the `city` outside?

TRY THIS · BROWSER CONSOLE

```
function printGrade(marks) {
    if (marks >= 50) {
        let grade = "Pass";
        console.log(grade);
    }
    console.log(grade);
}
printGrade(80);
console.log(grade);
```

Q7. What printed, and **which line** threw the error? Now change `let grade` to `var grade` and run it again. Which `console.log` works now, which one **still fails**, and why?

Part 2 — The Browser Object Model (BOM)

Concept — BOM & `window`. The browser gives JavaScript one **global object**, `window`. Everything the browser provides lives on it: `console`, `alert`, `prompt`, `location`, `document`, and more. Writing `window.` in front is **optional**.

TRY THIS · BROWSER CONSOLE

```
console.log(window);
```

TRY THIS · BROWSER CONSOLE

```
window.console.log("Hi");
console.log("Hi");
window.alert("Hello");
alert("Hello");
```

Q8. Why does each pair do **the same thing**?

TRY THIS · BROWSER CONSOLE

Run this twice — first type your name, then press **Cancel**.

```
let userName = prompt("Your name?");
alert("Hello " + userName);
```

Q9. What did `prompt` **return** each time?

TRY THIS · BROWSER CONSOLE

Open any tab, then run this in its console (**one by one**):

```
console.log(location.href);
console.log(location.hostname);
console.log(location.pathname);
console.log(location.search);
location.href = "https://zenithschool.ai";
```

Q10. Which part of the URL does each line print? What happened after the **last line**?

TRY THIS · BROWSER CONSOLE

```
console.log(document.title);
document.title = "My Page";
```

Q11. What printed, and what changed in the **browser tab**? (`document` is JavaScript's way into the page itself — that's Part 3.)

Part 3 — The Document Object Model (DOM)

Concept — DOM. The browser turns your HTML into a **tree of nodes**. `document` is the root node. Every HTML tag, and the text inside it, becomes a **node** that JavaScript can read and change.

TRY THIS · BROWSER CONSOLE

Replace the `<body>` of your Q1 `index.html` with this (keep the `script.js` link), then use the console on that page:

```
<h1 id="title">My Shop</h1>
<ul id="list">
    <li class="item">Pen</li>
    <li class="item">Notebook</li>
    <li class="item special">Bag</li>
</ul>
<p class="note">Prices in <b>INR</b></p>
```

TRY THIS · BROWSER CONSOLE

```
console.log(document.body.children);
console.log(document.body.children[0]);
console.log(document.body.children[1].children);
console.log(document.body.children[1].children[0]);
```

Q12. What did each line give? Why are there `<script>` elements among `body`'s children?

Concept — Selecting Elements.

- `getElementById("id")` — **one** element
- `getElementsByClassName("cls")` — a **collection** of elements
- `querySelector("css-selector")` — the **first** match
- `querySelectorAll("css-selector")` — **all** matches

TRY THIS · BROWSER CONSOLE

```
console.log(document.getElementById("title"));
console.log(document.getElementsByClassName("item"));
```

Q13. Which one returned a **single element**, and which returned a **collection**? What is the collection's `length`?

TRY THIS · BROWSER CONSOLE

```
console.log(document.querySelector(".item"));
console.log(document.querySelectorAll(".item"));
console.log(document.querySelector("#list .special"));
```

Q14. What did each line return? Why do these need `.` and `#` when `getElementById` doesn't?

TRY THIS · BROWSER CONSOLE

```
const title = document.getElementById("title");
title.style.color = "crimson";
title.style.backgroundColor = "lightyellow";
```

Q15. What changed on the page? Why do you think it is `backgroundColor` instead of `background-color`?

TRY THIS · BROWSER CONSOLE

```
const note = document.querySelector(".note");
console.log(note.innerText);
console.log(note.innerHTML);
note.innerText = "Prices in <i>USD</i>";
note.innerHTML = "Prices in <i>USD</i>";
```

Q16. Run the lines **one by one**. What is the difference between `innerText` and `innerHTML` — when **reading**, and when **writing**?

Q17. Using `querySelectorAll(".item")` and a **loop**, make every item's text color **blue**. Then change the `h1` text to `"My Shop (3 items)"` using `.length` (don't type 3 yourself).

Concept — Creating Elements. `document.createElement("tag")` makes a new element **in memory only** — it is not on the page yet. Fill it (e.g. `innerText`), then attach it with `parent.appendChild(el)`, which adds it as the **last child** of `parent`.

TRY THIS · BROWSER CONSOLE

Run the lines **one by one** on the same page:

```
const newItem = document.createElement("li");
newItem.innerText = "Eraser";
console.log(newItem);
const parent = document.getElementById("list");
parent.appendChild(newItem);
```

Q18. After **which line** did "Eraser" appear on the page, and why not earlier?
